08 - MQTT Real World IoT Connectivity - Session Notes

---

1. What Is MQTT and Why Does It Exist?

HTTP is the protocol your browser uses. You request a page. The server responds. Connection closes.

For IoT this model is terrible:

- Your ESP32 must poll the server constantly to get updates
- Server cannot push data to the device
- High overhead for tiny sensor readings
- Battery powered devices drain fast

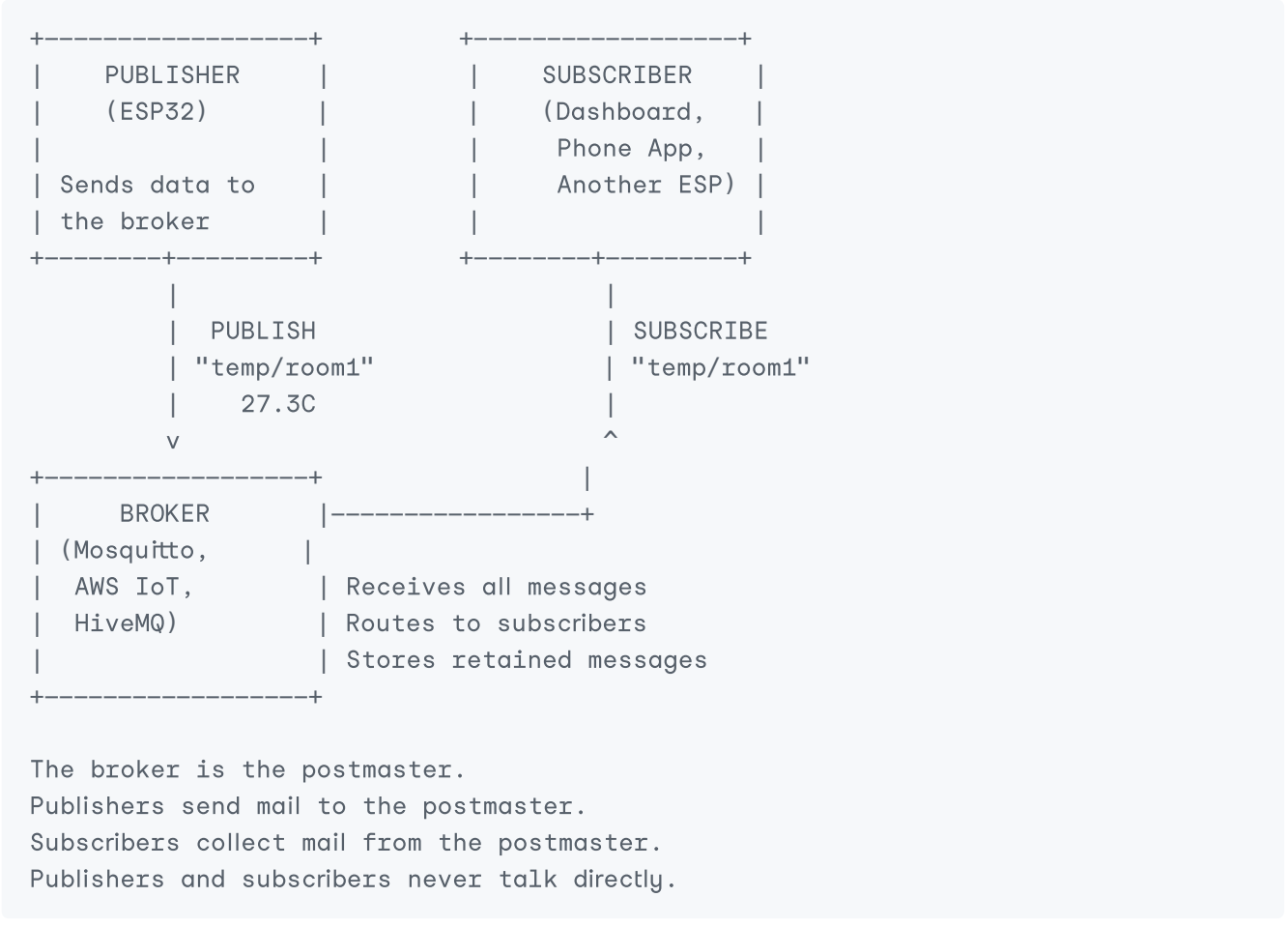
MQTT solves all of this.

PC Analogy — MQTT vs HTTP

```
HTTP MODEL (like a phone call):
+----------+          +----------+
| ESP32    |          | Server   |
+----------+          +----------+
| "Hey, any new data?" |         |
|--------------------> |         |
|                      | "No"    |
| <------------------- |         |
| [wait 10 seconds]    |         |
| "Hey, any new data?" |         |
|--------------------> |         |
|                      | "Yes!" |
| <------------------- |         |
Problem: ESP32 must keep asking. Wastes battery. Wastes bandwidth.

MQTT MODEL (like a WhatsApp group):
+----------+   +----------+   +----------+
| ESP32    |   | Broker   |   | Dashboard|
+----------+   +----------+   +----------+
ESP32 joins WhatsApp group "sensors"
Dashboard joins WhatsApp group "sensors"

ESP32 posts:  "Temperature: 27.3C"
              -> Broker receives it
              -> Broker forwards to Dashboard immediately
              -> No polling. No waiting. Instant delivery.

ESP32 posts:  "Temperature: 27.5C"
              -> Delivered instantly to all subscribers.
```

---

2. The Three Players — Broker, Publisher, Subscriber




```
The broker is the postmaster.
Publishers send mail to the postmaster.
Subscribers collect mail from the postmaster.
Publishers and subscribers never talk directly.
```

PC Analogy

```
MQTT Broker = WhatsApp Server
Topics      = WhatsApp Group chats
Publisher   = Person posting in the group
Subscriber  = Person reading the group

Just like WhatsApp:
- You can be in multiple groups (subscribe to multiple topics)
- You can post to multiple groups (publish to multiple topics)
- Messages are delivered even if you were briefly offline (QoS 1)
- The server handles routing — you never talk directly
```

---

3. Topics — How Data Is Organised

A topic is like a folder path for your messages.

```
Format:  level1/level2/level3

Examples:
  workshop/esp32/temperature
  workshop/esp32/humidity
  home/bedroom/temperature
```

```
  home/kitchen/humidity
  factory/line1/machine3/vibration
  factory/line1/machine3/temperature
```

Topic Wildcards

```
Single level wildcard: +
  workshop/+/temperature
  Matches: workshop/esp32/temperature
           workshop/raspberry/temperature
           workshop/arduino/temperature
  Does NOT match: workshop/esp32/sensors/temperature

Multi level wildcard: #
  workshop/#
  Matches: workshop/esp32/temperature
           workshop/esp32/humidity
           workshop/raspberry/temperature
           workshop/esp32/sensors/all/data
  # must be at the end of the topic
```

Good Topic Design for Production

```
GOOD topic structure:
  company/location/device_type/device_id/measurement

Example:
  analogdata/bengaluru/esp32/device001/temperature
  analogdata/bengaluru/esp32/device001/humidity
  analogdata/bengaluru/esp32/device001/status

Benefits:
  - Subscribe to all devices: analogdata/bengaluru/esp32/+/temperature
  - Subscribe to one device:  analogdata/bengaluru/esp32/device001/#
  - Subscribe to all data:    analogdata/bengaluru/#

BAD topic design:
  data          <- too generic, no hierarchy
  ESP32_TEMP    <- not extensible, no location
  /temperature  <- starts with / — technically valid but confusing
```

---

4. QoS — Quality of Service Levels

QoS determines how reliably a message is delivered.

```
QoS 0 — At Most Once (Fire and Forget)
+--------+          +--------+
| Sender |          | Broker |
+--------+          +--------+
  PUBLISH --------->
```

```
                      Done. No confirmation.
                      If lost — lost forever.

Use for: High frequency sensor data (temperature every second)
         Losing one reading is acceptable
         Battery powered devices — lowest overhead

QoS 1 — At Least Once (Confirmed Delivery)
+--------+          +--------+
| Sender |          | Broker |
+--------+          +--------+
  PUBLISH --------->
          <--------- PUBACK (received)
  If no PUBACK — sender retries
  Message delivered at least once — may arrive twice

Use for: Important readings where loss is not acceptable
         Commands to devices (turn LED on)
         Most production IoT use cases

QoS 2 — Exactly Once (Guaranteed Single Delivery)
+--------+          +--------+
| Sender |          | Broker |
+--------+          +--------+
  PUBLISH --------->
          <--------- PUBREC
  PUBREL  --------->
          <--------- PUBCOMP
  Four-step handshake. Highest overhead.
  Message arrives exactly once — never duplicated

Use for: Financial transactions, billing meters
         Cases where duplicate processing is dangerous
         Rarely needed in typical IoT
```

QoS Decision Table

Data type	QoS	Reason
Temperature every second	0	Losing one reading is fine
Critical alert	1	Must not be lost
LED on/off command	1	Device must receive it
Payment event	2	Must not be processed twice
Heartbeat / status	0	High frequency, loss acceptable
Configuration update	1	Device must apply it

---

5. MQTT in ESP-IDF — Full Setup

Step 1 — Add MQTT to CMakeLists.txt

```
idf_component_register(
    SRCS "main.c"
    INCLUDE_DIRS "."
    REQUIRES
        driver
        esp_wifi
        mqtt
        nvs_flash
        esp_netif
        esp_event
        json
        log
)
```

Step 2 — MQTT Event Handler

```
#include "mqtt_client.h"
#include "esp_log.h"

static const char *TAG = "mqtt";

static esp_mqtt_client_handle_t mqtt_client = NULL;
static bool mqtt_connected = false;

static void mqtt_event_handler(void *handler_args,
                               esp_event_base_t base,
                               int32_t event_id,
                               void *event_data)
{
    esp_mqtt_event_handle_t event = event_data;

    switch (event->event_id) {

        case MQTT_EVENT_CONNECTED:
            mqtt_connected = true;
            ESP_LOGI(TAG, "Connected to broker");

            /* Subscribe to command topics after connecting */
            esp_mqtt_client_subscribe(mqtt_client,
                "workshop/esp32/commands/#", 1);
            ESP_LOGI(TAG, "Subscribed to commands");
            break;

        case MQTT_EVENT_DISCONNECTED:
            mqtt_connected = false;
            ESP_LOGW(TAG, "Disconnected from broker — will reconnect");
            /* ESP-IDF MQTT client reconnects automatically */
            break;

        case MQTT_EVENT_DATA:
            /* Incoming message received */
```

```
            ESP_LOGI(TAG, "Topic: %.*s",
                     event->topic_len, event->topic);
            ESP_LOGI(TAG, "Data:  %.*s",
                     event->data_len, event->data);

            /* Handle commands */
            if (strncmp(event->topic,
                        "workshop/esp32/commands/led",
                        event->topic_len) == 0) {
                if (strncmp(event->data, "on", event->data_len) == 0) {
                    gpio_set_level(GPIO_NUM_2, 1);
                    ESP_LOGI(TAG, "LED turned ON by command");
                } else {
                    gpio_set_level(GPIO_NUM_2, 0);
                    ESP_LOGI(TAG, "LED turned OFF by command");
                }
            }
            break;

        case MQTT_EVENT_ERROR:
            ESP_LOGE(TAG, "MQTT error — type: %d",
                     event->error_handle->error_type);
            break;

        case MQTT_EVENT_PUBLISHED:
            ESP_LOGD(TAG, "Message published — msg_id: %d",
                     event->msg_id);
            break;

        case MQTT_EVENT_SUBSCRIBED:
            ESP_LOGI(TAG, "Subscribed — msg_id: %d", event->msg_id);
            break;

        default:
            break;
    }
}
```

Step 3 — MQTT Client Init

```
static void mqtt_init(void)
{
    esp_mqtt_client_config_t mqtt_cfg = {
        .broker = {
            .address = {
                .uri = "mqtt://test.mosquitto.org:1883",
            },
        },
        .credentials = {
            .client_id = "analog_data_esp32_001",
        },
        .session = {
            .keepalive = 60,          /* Send ping every 60 seconds */
            .disable_clean_session = false,
```

```
        },
        .network = {
            .reconnect_timeout_ms = 5000,   /* Reconnect every 5s */
            .timeout_ms = 10000,            /* Connection timeout */
        },
    };

    mqtt_client = esp_mqtt_client_init(&mqtt_cfg);

    ESP_ERROR_CHECK(esp_mqtt_client_register_event(
        mqtt_client,
        ESP_EVENT_ANY_ID,
        mqtt_event_handler,
        NULL
    ));

    ESP_ERROR_CHECK(esp_mqtt_client_start(mqtt_client));

    ESP_LOGI(TAG, "MQTT client started");
}
```

---

6. Publishing Sensor Data

Plain Text Payload (Simple)

```
static void publish_temperature(float temperature)
{
    char payload[32];
    snprintf(payload, sizeof(payload), "%.2f", temperature);

    int msg_id = esp_mqtt_client_publish(
        mqtt_client,
        "workshop/esp32/sensors/temperature",
        payload,
        0,      /* Payload length — 0 = use strlen */
        1,      /* QoS 1 */
        0       /* Retain = false */
    );

    if (msg_id < 0) {
        ESP_LOGE(TAG, "Publish failed");
    } else {
        ESP_LOGI(TAG, "Published %.2f C (msg_id=%d)",
                 temperature, msg_id);
    }
}
```

JSON Payload (Production Standard)

JSON is the standard payload format because:

- Self-describing — field names are included
- Easy to parse on any platform
- Extensible — add new fields without breaking existing subscribers
- Readable by humans for debugging

```c
#include "cJSON.h"

static void publish_sensor_json(float temperature, float humidity)
{
    /* Build JSON object */
    cJSON *root = cJSON_CreateObject();
    if (root == NULL) {
        ESP_LOGE(TAG, "JSON creation failed — out of memory");
        return;
    }

    cJSON_AddStringToObject(root, "device_id", "esp32_001");
    cJSON_AddNumberToObject(root, "temperature", temperature);
    cJSON_AddNumberToObject(root, "humidity", humidity);
    cJSON_AddNumberToObject(root, "timestamp",
                            (double)esp_timer_get_time() / 1000000);

    /* Serialise to string */
    char *json_str = cJSON_PrintUnformatted(root);
    if (json_str == NULL) {
        ESP_LOGE(TAG, "JSON serialise failed");
        cJSON_Delete(root);
        return;
    }

    /* Publish */
    esp_mqtt_client_publish(
        mqtt_client,
        "workshop/esp32/sensors/all",
        json_str,
        strlen(json_str),
        1,    /* QoS 1 */
        0     /* Not retained */
    );

    ESP_LOGI(TAG, "Published: %s", json_str);

    /* Always free cJSON memory */
    cJSON_free(json_str);
    cJSON_Delete(root);
}
```

Example JSON Payload

```json
{
  "device_id": "esp32_001",
  "temperature": 27.30,
  "humidity": 64.50,
```

```
  "timestamp": 1234567.89
}
```

---

7. Subscribing to Commands

An ESP32 is not just a data sender. It also receives commands.

```
Dashboard publishes: workshop/esp32/commands/led  -> "on"
ESP32 receives it -> turns LED on

Dashboard publishes: workshop/esp32/commands/interval -> "5000"
ESP32 receives it -> changes publish interval to 5 seconds
```

Parsing Incoming Commands

```
case MQTT_EVENT_DATA: {
    /* Make null-terminated copies of topic and data */
    char topic[128] = {0};
    char data[256]  = {0};

    snprintf(topic, sizeof(topic), "%.*s",
             event->topic_len, event->topic);
    snprintf(data, sizeof(data), "%.*s",
             event->data_len, event->data);

    ESP_LOGI(TAG, "Command received — topic: %s  data: %s",
             topic, data);

    /* LED control command */
    if (strcmp(topic, "workshop/esp32/commands/led") == 0) {
        if (strcmp(data, "on") == 0) {
            gpio_set_level(GPIO_NUM_2, 1);
        } else if (strcmp(data, "off") == 0) {
            gpio_set_level(GPIO_NUM_2, 0);
        }
    }

    /* Publish interval command */
    if (strcmp(topic, "workshop/esp32/commands/interval") == 0) {
        int interval = atoi(data);
        if (interval >= 1000 && interval <= 60000) {
            publish_interval_ms = interval;
            ESP_LOGI(TAG, "Publish interval changed to %dms", interval);
        }
    }
    break;
}
```

---

8. Last Will and Testament — Detecting Dead Devices

The Last Will and Testament (LWT) is a message the broker publishes automatically when your device disconnects unexpectedly.

PC Analogy

```
LWT = An automated Out of Office email

You tell the email server:
"If I stop responding for more than 60 seconds,
automatically send this message to my team:
'Rajath is offline — please check his computer'"

When your ESP32 goes offline unexpectedly —
the broker sends your pre-configured message.
Your dashboard knows the device is offline.
```

LWT Configuration

```
esp_mqtt_client_config_t mqtt_cfg = {
    .broker = {
        .address = {
            .uri = "mqtt://test.mosquitto.org:1883",
        },
    },
    .session = {
        .last_will = {
            .topic   = "workshop/esp32/status",
            .msg     = "{\"status\":\"offline\",\"device\":\"esp32_001\"}",
            .msg_len = 0,   /* 0 = use strlen */
            .qos     = 1,
            .retain  = 1,   /* Retain so dashboard always sees it */
        },
        .keepalive = 60,
    },
};
```

Publishing Online Status on Connect

```
case MQTT_EVENT_CONNECTED:
    /* Publish online status — overrides the LWT offline message */
    esp_mqtt_client_publish(
        mqtt_client,
        "workshop/esp32/status",
        "{\"status\":\"online\",\"device\":\"esp32_001\"}",
        0,
        1,
        1    /* Retain = true — dashboard always sees latest status */
    );
    break;
```

The Full Lifecycle

```
Device powers on
        |
        v
Device connects to broker
        |
        v
Device publishes: status = "online" (retained)
        |
        v
Device publishes sensor data every 2 seconds
        |
        v
Device loses power unexpectedly (no disconnect message)
        |
        v
Broker waits 1.5x keepalive (90 seconds)
        |
        v
Broker publishes LWT: status = "offline" (retained)
        |
        v
Dashboard shows device as offline
        |
        v
Alert sent to engineer
```

---

9. Retained Messages

A retained message is stored by the broker and immediately sent to any new subscriber.

```
Without retained message:
  New dashboard opens -> subscribes to status topic
  Waits... waits... waits...
  No message until next publish (could be 60 seconds)
  Dashboard shows "unknown" for 60 seconds

With retained message:
  New dashboard opens -> subscribes to status topic
  Broker immediately delivers the LAST retained message
  Dashboard shows current status instantly
```

```
/* Publish with retain = 1 */
esp_mqtt_client_publish(
    mqtt_client,
    "workshop/esp32/status",
    "online",
    0,
    1,
```

```
    1   /* retain = 1 */
);
```

When to Use Retained Messages

Use retained	Do not use retained
Device online/offline status	High frequency sensor data
Last known sensor value	Log messages
Device configuration	Commands sent to device
Software version	Heartbeats

---

10. MQTT vs HTTP — When to Use Which

Property	MQTT	HTTP
Protocol	TCP based, persistent	TCP based, request-response
Connection	Stays open	Opens and closes per request
Direction	Bidirectional	Request then response
Overhead	Very low (2 byte header)	High (headers, cookies)
Push data to device	Yes — subscribe	No — device must poll
Battery friendly	Yes	No
Cloud support	AWS IoT, Azure, GCP	Everything
Best for	Real time sensor data, commands	REST APIs, file downloads
Worst for	Large file transfer	Real time bidirectional

Production rule: Use MQTT for sensor data and device commands. Use HTTP for firmware OTA updates and API calls.

---

11. Production MQTT Checklist

```
Before shipping any IoT product with MQTT:

Security:
[ ] Use TLS (port 8883) — never plain MQTT in production
[ ] Use unique client ID per device
[ ] Use username/password or client certificates
[ ] Never hardcode broker credentials in firmware

Reliability:
```

```
[ ] Use QoS 1 for important data
[ ] Configure Last Will and Testament
[ ] Handle MQTT_EVENT_DISCONNECTED gracefully
[ ] ESP-IDF auto-reconnects — verify this works

Design:
[ ] Use structured JSON payloads
[ ] Use hierarchical topic structure
[ ] Use retained messages for status and last known values
[ ] Publish device metadata (firmware version, IP) on connect

Monitoring:
[ ] Log publish failures
[ ] Log subscription failures
[ ] Monitor message delivery rate
[ ] Alert on extended offline periods
```

Summary Table

Concept	What it is	PC / Real world equivalent
MQTT	Lightweight IoT messaging protocol	WhatsApp for devices
Broker	Central message router	WhatsApp server
Publisher	Sends messages to broker	Person posting in group
Subscriber	Receives messages from broker	Person reading the group
Topic	Message address and category	WhatsApp group name
QoS 0	Fire and forget	WhatsApp message (no read receipt)
QoS 1	Confirmed delivery	WhatsApp with blue ticks
QoS 2	Exactly once	WhatsApp + confirmation call
Retained message	Broker stores last message	Pinned message in group
LWT	Auto message on unexpected disconnect	Out of office reply
Keepalive	Periodic ping to keep connection alive	WhatsApp typing indicator
Clean session	Start fresh with no subscriptions	New WhatsApp install
Client ID	Unique device identifier	Your phone number
JSON payload	Structured data format	A formatted report
TLS	Encrypted MQTT connection	WhatsApp encryption
